HydraPQ-v1

A Hybrid Post-Quantum Authenticated Key Exchange and Record Protocol

Author: Justin Fairchild
Version: 0.1
Status: Experimental draft. No security claims beyond conjecture. Not production safe.

Abstract

We present **HydraPQ-v1**, a hybrid authenticated key exchange (AKE) protocol combining a post-quantum key encapsulation mechanism (KEM) and a classical elliptic-curve KEM, followed by a transcript-bound key derivation and an AEAD record layer.

HydraPQ-v1 targets:

- Hybrid classical + post-quantum confidentiality
- Forward secrecy
- Transcript binding
- Replay resistance
- Rekey capability

Security claims are informal and reduction sketches are incomplete. Cryptanalysis is invited.

---

1. Notation

Let:

- || denote concatenation
- H(x) denote SHA-256
- HKDF-Extract, HKDF-Expand per RFC 5869
- AEAD.Enc, AEAD.Dec denote either:
    - ChaCha20-Poly1305
    - AES-256-GCM

Let:

- KEM_PQ = (KeyGen_pq, Encaps_pq, Decaps_pq)
- KEM_EC = (KeyGen_ec, Encaps_ec, Decaps_ec)

All randomness is assumed uniform and cryptographically secure.

---

2. Design Goals

HydraPQ-v1 aims to provide:

1. **Confidentiality** under IND-CCA assumptions of both KEMs
2. **Authentication** via static signature keys
3. **Forward secrecy** via ephemeral keys
4. **Hybrid resilience** requiring break of at least one component
5. **Transcript binding**
6. **Replay protection** at the record layer

Out of scope:

- Side-channel resistance
- Fault injection
- Formal proofs
- Implementation hardening

---

3. Adversary Model

We assume a network adversary with:

- Full message interception and modification capability
- Adaptive chosen-ciphertext access
- Long-term key compromise after session completion
- Computational bounds defined by security parameters

We do not assume constant-time execution.

---

4. Long-Term Key Generation

Each party generates:

- (sk_pq, pk_pq) ← KeyGen_pq()
- (sk_ec, pk_ec) ← KeyGen_ec()
- (sk_sig, pk_sig) for digital signatures

Public identity:

ID = (pk_pq, pk_ec, pk_sig)

---

5. Handshake Protocol

5.1 Ephemeral Key Generation

Client generates:

(ep_sk_pq_C, ep_pk_pq_C)
(ep_sk_ec_C, ep_pk_ec_C)
n_C ← {0,1}^96

Server generates:

(ep_sk_pq_S, ep_pk_pq_S)
(ep_sk_ec_S, ep_pk_ec_S)
n_S ← {0,1}^96

5.2 Message Flow

Message 1 (Client → Server)

ep_pk_pq_C
ep_pk_ec_C
n_C

Message 2 (Server → Client)

```
ct_pq = Encaps_pq(ep_pk_pq_C)
ct_ec = Encaps_ec(ep_pk_ec_C)
n_S
σ_S = Sign(sk_sig_S, transcript_1)
```

where:

```
transcript_1 = ep_pk_pq_C || ep_pk_ec_C || n_C || ct_pq || ct_ec || n_S
```

Message 3 (Client → Server)

Client verifies σ_S.

Client computes:

```
ss_pq = Decaps_pq(ep_sk_pq_C, ct_pq)
ss_ec = Decaps_ec(ep_sk_ec_C, ct_ec)
```

Client responds:

```
σ_C = Sign(sk_sig_C, transcript_1 || σ_S)
```

Server verifies σ_C.

Server computes:

```
ss_pq = Decaps_pq(ep_sk_pq_S, ct_pq)
ss_ec = Decaps_ec(ep_sk_ec_S, ct_ec)
```

---

6. Hybrid Secret Derivation

Define:

```
hybrid_secret = ss_pq || ss_ec
```

Transcript hash:

```
th = H(ep_pk_pq_C || ep_pk_ec_C || n_C ||
       ct_pq || ct_ec || n_S || σ_S || σ_C)
```

Master key derivation:

```
prk = HKDF-Extract(
        salt = H("HydraPQ-v1" || th),
```

```
    IKM  = hybrid_secret
  )

master = HKDF-Expand(prk, "Hydra master", 32)
```

---

7. Record Layer: HydraCipher-v1

7.1 Key Schedule

From prk derive:

```
k_app      = HKDF-Expand(prk, "Hydra app key",   32)
nonce_base = HKDF-Expand(prk, "Hydra nonce",     12)
k_rekey    = HKDF-Expand(prk, "Hydra rekey",     32)
```

Each direction maintains independent state.

7.2 Sequence Number

Each sender maintains:

seq ∈ {0,…,2^64−1}

7.3 Nonce Construction

For each record:

nonce = nonce_base XOR (0x00000000 || uint64_be(seq))

Nonce reuse under same k_app is forbidden.

7.4 Record Structure

```
struct HydraRecord {
   uint8   version;
   uint8   flags;
   uint64  seq;
   uint16  aad_len;
   byte    aad[aad_len];
```

```
   uint32  ct_len;
   byte    ciphertext[ct_len];
}
```

Ciphertext includes AEAD tag.

7.5 Encryption

```
ciphertext = AEAD.Enc(
             key   = k_app,
             nonce = nonce,
             plaintext,
             aad
           )
seq++
```

7.6 Decryption

```
plaintext = AEAD.Dec(
             key   = k_app,
             nonce = nonce,
             ciphertext,
             aad
          )
```

Reject on failure.

Replay policy: reject duplicate seq.

---

8. Rekey Procedure

After N records:

```
prk2 = HKDF-Extract(
        salt = H("Hydra rekey" || th),
        IKM  = k_rekey || uint64_be(seq)
      )

k_app      = HKDF-Expand(prk2, "Hydra app key", 32)
```

```
nonce_base = HKDF-Expand(prk2, "Hydra nonce",   12)
k_rekey    = HKDF-Expand(prk2, "Hydra rekey",   32)
```

---

9. Security Assumptions

HydraPQ-v1 relies on:

1. IND-CCA security of KEM_PQ
2. Hardness of elliptic-curve discrete logarithm
3. Collision resistance of SHA-256
4. PRF security of HMAC (HKDF)
5. AEAD confidentiality + integrity

Breaking session secrecy requires breaking at least one KEM or HKDF extraction.

---

10. Informal Reduction Sketch

Suppose an adversary distinguishes master from random.

Then either:

- HKDF output is distinguishable
- At least one shared secret is distinguishable
- Or transcript binding fails

Assuming HKDF acts as a randomness extractor, distinguishing master implies distinguishing at least one component secret.

Therefore, compromise requires breaking either PQ KEM or EC KEM.

Formal proof omitted.

---

11. Known Open Questions

- Is concatenation optimal for hybrid mixing?
- Does transcript binding fully prevent downgrade?

- Is KCI resistance complete?
- Does rekey provide meaningful state-compromise resilience?
- Are there subtle reflection or replay edges?

---

12. Disclaimer

HydraPQ-v1 is experimental.

No formal proof is provided.
No side-channel guarantees exist.
No deployment recommendation is made.

Cryptanalysis invited.